

1.1 What is Encapsulation? Explain how Java achieves it using access modifiers and getter/setter methods. Give one real-life analogy.

Encapsulation - is an important concept of Object-Oriented Programming (OOP). It means wrapping data and methods together inside a single class and protecting the data from direct access from outside the class.

In Java, encapsulation is mainly achieved by -

1. Declaring data members as private.
2. Providing public getter and setter methods to access or modify the private data.

For example:

```
class Student {  
    private String name;  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

Here, name is private, so it cannot be directly accessed

from outside the class. The `getName()` method is used to read the value, and `setName()` is used to change the value.

Access Modifiers -

- `private` → accessible only inside the same class.
- `public` → accessible from anywhere.
- `protected` → accessible within the same package and subclass.
- `default` → accessible within the same package.

For proper encapsulation, we usually keep data private and provide controlled access through public methods.

Real Life analogy -

A bank account is a good example of encapsulation. We cannot directly change the money stored in our bank account. Instead, we use controlled operations such as deposit and withdraw. The actual balance is protected by the bank.

In short, Encapsulation provides data hiding, security, and controlled access to data.

1.2. What is Polymorphism? Distinguish compile-time (static) vs runtime (dynamic) polymorphism with one example each.

Polymorphism - is one of the main concepts of Object-Oriented Programming. In Java, polymorphism means the same method name or interface can perform different behaviors depending on how it is used.

There are two main types of Polymorphism in Java:

1. Compile-time Polymorphism: Compile time polymorphism is also called static polymorphism. It is mainly achieved through method overloading.

In method overloading, multiple methods have the same name but different parameters -

Example:

```
class Calculator {
    int add (int a, int b) {
        return a+b;
    }
    double add (double a, double b){
        return a+b;
    }
}
```

2. Run-time Polymorphism - Run-time polymorphism is also called Dynamic Polymorphism. It is achieved through method overriding.

Example :

```
class Animal {
    void sound () {
        system.out.println ("Animal makes sound");
    }
}
class Dog extends Animal {
    @Override
    void sound () {
        system.out.println ("Dog barks.");
    }
}
```

If we write :

```
Animal a = new Dog();
a.sound();
```

The output will be: Dog barks .

Difference -

Compile-time	Run-time
Also call static Polymorphism	Also called Dynamic Polymorphism
Achieved by method overloading	Achieved by method overriding
Decision occurs at compile time	Decision occurs at Run-time
Same method name, diff. parameter.	Same method with redefined behavior.

1.3 Write a BankAccount class demonstrating encapsulation (Private [balance + get balance () / deposit ()]) and Polymorphism (overloaded deposit (double) / deposit (double, string remarks)). Include a Main class calling both overloads.

```
class BankAccount {
    private double balance ;
    // constructor
    public BankAccount (double balance) {
        this.balance = balance ;
    }
    // getter
    public double getBalance () {
        return balance ;
    }
    // Deposit method - 1
    public void deposit (double amount) {
        balance = balance + amount ;
        System.out.println ("Deposited: " + amount);
    }
    // Deposit method - 2 (overloaded)
    public void deposit (double amount, String remarks) {
        balance = balance + amount ;
        System.out.println ("Deposited: " + amount);
        System.out.println ("Remarks: " + remarks);
    }
}
```

```

public class Main {
    public static void main (String[] args) {
        BankAccount account = new BankAccount (5000);
        // 1st overloaded method call
        account.deposit(1000);
        // 2nd overloaded method call
        account.deposit(2000, "Salary");
        System.out.println ("Final Balance: " + account.getBalance());
    }
}

```

This program demonstrates both encapsulation and polymorphism.

Encapsulation - `private double balance;`

the balance variable is private, so it cannot be directly accessed from outside the `BankAccount` class.

We use - `public double getBalance()` - to safely read the balance

Polymorphism - There are two deposit methods -

1. deposit (double amount)
2. deposit (double amount, String remarks)

2.1. Compare overloading vs overriding : definition, class involved, parameters, return type, binding time.

Method Overloading - means having methods with the same name but different parameter lists in the same class. It is an example of compile-time polymorphism.

Method Overriding - a child class provides its own implementation of a method that is already defined in its parent class. It is an example of ~~compile-time~~ run-time polymorphism.

Overloading	Overriding
Same method name with diff. parameters.	Same method name and same parameters.
Occurs within same class	Requires parent and child class
Parameter must be different	Parameter must be same
Compile-time binding	Runtime binding
Static Polymorphism	Dynamic Polymorphism

Example: Overloading

```
void show (int x) { }  
void show (String x) { }
```

Overriding

```
class Animal {  
    void sound () {  
        System.out.println("Animal sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound () {  
        System.out.println("Dog barks");  
    }  
}
```

2.2 Explain Early binding vs late binding. Why is overriding resolved at run time and overloading at compile time?

Binding means connecting a method call with the actual method implementation.

Early Binding - is also called static/compile-time binding when the compiler decides which method should be called before the program runs.

Method Overloading uses early binding because compiler can identify the correct method from the method name and parameter list.

Example -

```
class Calculator {  
    void add (int a, int b) {  
        System.out.println (a+b);  
    }  
    void add (double a, double b) {  
        System.out.println (a+b);  
    }  
}
```

Late Binding - also called dynamic/run-time binding. It is the actual method to execute is decided during program execution.

Method Overriding uses late binding because the actual object may be a child-class object even

when the reference is of the parent type.

Example -

```
Animal a = new Dog();  
a.sound();
```

Here, a is an Animal reference, but the actual object is Dog. Therefore, Java calls Dog's sound() method at run-time.

2.3 Shape with area(), overridden by Circle and Rectangle (late binding via shape ref.). Add overloaded describe/str / describe(String, int) in shape (early binding), show example output.

```
class Shape {  
    public void area() {  
        System.out.println("Area of Shape");  
    }  
  
    public void describe(String name) {  
        System.out.println("Shape: " + name);  
    }  
  
    public void describe(String name, int sides) {  
        System.out.println("Shape: " + name);  
        System.out.println("Number of sides: " + sides);  
    }  
}  
  
class Circle extends Shape {  
    @Override  
    public void area() {  
        double r = 5;  
        System.out.println("Circle Area: " + Math.PI  
            * r * r);  
    }  
}  
  
class Rectangle extends Shape {  
    @Override  
    public void area() {  
        double length = 5,
```

```

double width = 4;
System.out.println ("Rectangle Area: " + length
                    * width);
}
}

public class main {
    public static void main (String[] args) {
        shape s1 = new Circle();
        shape s2 = new Rectangle();

        // late binding
        shape s1.area();
        s2.area();

        // Early binding
        s1.describe("Circle");
        s2.describe("Rectangle", 4);
    }
}

```

Sample Output -

Circle Area: 78.53981633974483

Rectangle Area: 20.0

Shape : Circle

Shape : Rectangle

Number of Sides : 4

COMPLEX VARIABLE

1

Taylor's Theorem - If a function $f(z)$ is analytic at all the points inside a circle C , with its centre at the point a and radius R , then at each point z inside C .

$$f(z) = \sum_{n=0}^{\infty} a_n (z-a)^n \quad \text{where} \quad a_n = \frac{f^n(a)}{n!}$$

$$\text{i.e. } f(z) = \sum_{n=0}^{\infty} \frac{f^n(a) (z-a)^n}{n!}$$

3.1 Define abstract class and interface. List 24 structural differences fields, constructors, method bodies, multiple inheritance)

Abstract Class - is a class declared using **abstract** keyword. It can contain both abstract method and concrete method.

Example:

```
abstract class Vehicle {
    String brand ;
    Vehicle (String brand) {
        this.brand = brand ;
    }
    void showBrand () {
        System.out.println ("Brand" + brand);
    }
    abstract void start ();
}

class Car extends Vehicle {
    Car (String brand) {
        super (brand);
    }
    void start () {
        System.out.println ("Car starts with a key");
    }
}

public class Main {
    public static void main (String[] args) {
        Car c = new Car ("Toyota");
        c.showBrand();
        c.start();
    }
}
```

Interface - is a contract that specifies what a class should do. A class uses the implements keyword to implement interface.

Example -

```
interface Vehicle {
    void start ();
}

class Car implements Vehicle {
    public void start () {
        system.out.println ("Car starts with a key");
    }
}

public class Main {
    public static void main (String[] args) {
        Car c = new Car ();
        c.start ();
    }
}
```

Difference -

	Abstract Class	Interface
Fields	Can have instance, static mutable or final fields	Fields are implicitly public static final (const. only)
Constructor	Can have constructor to initialize object state.	Can't have constructor
Method bodies	both abstract (without body) or concrete (with body) methods.	Traditionally only abstract method. Though modern version allow default and static method.
Multiple inheritance	A class can extend only one abstract class (single inheritance)	Can implement multiple inheritance.

3.2 When should you choose an abstract class vs interface? Give two real world scenarios ("is-a" with shared code vs "can-do" capability)

Abstract class - we choose when related classes have:

- A common parent
- Common fields
- Common methods / code.
- An "is-a" relationship.

Real-world example:

Car, Bus and Truck are all Vehicles

So, the Vehicle class can provide common code such as startEngine().

Interface - we choose when different classes need to have the same capability, even if they are not closely related.

Real-world example -

Car → Insurable

House → Insurable

Phone → Insurable ; they are not same type of object, but they can have an Insurable capability

In short:

• Abstract → "What you are" / is-a

• Insurance → "What you can do" / can-do / has-a

3.3 Vehicle (abstract, startEngine() implemented + abstract fuelType()) and Insurable (interface, calculatePremium()). Car extends Vehicle and implements Insurable. Comment on why each construct was chosen.

```

abstract class Vehicle {
    public void startEngine() { // concrete method
        System.out.println ("Engine started.");
    }
    public abstract void fuelType(); // Abstract method
                                    (no body)
}

interface Insurable {
    void calculatePremium();
}

class Car extends Vehicle implements Insurable {
    @Override
    public void fuelType() {
        System.out.println ("Fuel Type : Petrol");
    }
    @Override
    public void calculatePremium() {
        System.out.println ("Insurance premium: 5000");
    }
}

```

```

public class Main {
    public static void main (String[] args) {
        Car car = new Car ();
        car.startEngine ();
        car.fuelType ();
        car.calculatePremium ();
    }
}

```

Output-

Engine started.

Fuel Type : Petrol

Insurance Premium : 5000

Why abstract class -

Car is a type of Vehicle, so there is an is-a relationship.

Vehicle also provides common behavior:

startEngine()

while forcing subclasses to define:

fuelType().

Why interface — Insurable represents capability.

A car can be insured, so Car implements Insurable

so

Car is a Vehicle + Car has Insurable capability

4.1 Compare ArrayList, Vector, LinkedList : underlying structure, synchronization, insert/delete vs random access performance.

All three are classes in the Java Collection Framework used to store multiple objects.

Feature	ArrayList	Vector	LinkedList
Structure	Dynamic Array	Dynamic Array	Doubly linked list
Synchronization	Not synchronized	Synchronized	Not synchronized
Random access	Fast, $O(1)$	Fast, $O(1)$	Slow, $O(n)$
Insert/delete middle	Relatively slower	Relatively slower	faster if node position is known
Performance	Generally fast	Slower due to synchronization	Good for frequent insertion/deletion
Modern Usage	Very common	Less common	Common for linked list operations

ArrayList - uses a dynamic array. It is good when we frequently access elements using indexes.

```
ArrayList<String> list = new ArrayList<>();
```

Vector - is also a dynamic array but its methods are synchronized. Therefore, it is generally slower than ArrayList.

LinkedList - stores elements using linked nodes. Insertion and deletion can be efficient, but random access is slower.

4.2 Explain set and its implementations HashSet, LinkedHashSet, TreeSet. How does TreeSet maintain order?

Set - is a collection that does not allow duplicate elements.

```
Set<String> names = new HashSet<>();
```

The major implementations are -

1. HashSet -

- Does not allow duplicates.
- Does not guarantee insertion order.
- Uses hashing internally.
- Usually provides fast operations.

```
HashSet<String> set = new HashSet<>();
```

2. LinkedHashSet -

- Does not allow duplicates
- Maintains insertion order
- Uses hashing with linked structure

```
LinkedHashSet<String> set = new LinkedHashSet<>();
```

3. TreeSet -

- Does not allow duplicates.
- Maintains elements in sorted order.
- Uses a tree-based structure
- Normally uses natural ordering or a supplied Comparator.

```
TreeSet<String> set = new TreeSet<>();
```

Comparison -

set	Duplicate	Order
HashSet	No	No guaranteed order
Linked HashSet	No	Insertion order
TreeSet	No	Sorted order

How does TreeSet maintain order -

TreeSet is based on a balanced search tree (Red-Black tree) and keeps elements sorted according to their natural ordering or Comparator.

4.3 store 5 student names in an ArrayList (print with enhanced for-loop) and in a TreeSet (print, observe sorting). Comment on ordering difference.

```
import java.util. ArrayList ;
import java.util. TreeSet ;

public class main {
    public static void main (String[] args) {
        ArrayList <String> students = new ArrayList <>();

        students.add ("Rahim");
        students.add ("Karim");
        students.add ("Rubi");
        students.add ("Rahim");
        students.add ("Tania");

        System.out.println ("ArrayList:");
        for (String name : students) {
            System.out.println (name);
        }

        TreeSet <String> sortedStudents = new TreeSet <>();
        sortedStudents.add ("Rahim");
        sortedStudents.add ("Karim");
        sortedStudents.add ("Rubi");
        sortedStudents.add ("Rahim");
        sortedStudents.add ("Tania");
```

```
system.out.println("\nTreeSet:");  
for (String name : sortedStudents) {  
    system.out.println(name);  
}
```

Output -

ArrayList :

Rahim
karim
Rubi
Raham
Nadda

TreeSet :

Rahim
karim
Rubi
Raham
Nadia.

5.1 List 2.3 ways to implement multithreading in Java (extends Thread, implements Runnable, ExecutorService/Callable). Which is preferred and why?

Multithreading - means executing multiple threads concurrently within a program.

There are several ways to create/manage threads in Java.

1. Extending Thread - create a class that extends Thread and override run().

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running");
    }
}
```

2. Implementing Runnable - create a class implementing

Runnable.

```
class MyTask implements Runnable {
    public void run() {
        System.out.println("Task running");
    }
}

public class Main {
    public static void main (String[] args) {
        new Thread (new MyTask()).start();
    }
}
```

3. ExecutorService - manages a pool of threads and is useful for larger applications.

```
class MyTask implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println (Thread.currentThread().getName() +
                                " is running: " + i);
        }
    }
    try {
        Thread.sleep(500);
    } catch (InterruptedException e) {
        System.out.println ("Thread was interrupted");
    }
}
```

Callable - is useful when a thread needs to return a result or throw an exception.

Which preferred - For simple tasks, Runnable is often preferred over extending Thread because Java supports single inheritance. If we extend Thread we cannot extend another class.

For larger applications, ExecutorService is generally preferred because it manages thread creation and reuse efficiently.

5.2 Two Threads printing 1-5 concurrently - one via extends Thread, one via implements Runnable. Show how both are started from main().

```

class ThreadOne extends Thread {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread One: " + i);
        }
    }
}

class ThreadTwo implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread Two: " + i);
        }
    }
}

public class Main {
    public static void main (String[] args) {
        ThreadOne t1 = new ThreadOne(),
        ThreadTwo task = new ThreadTwo();
        ThreadTwo t2 = new Thread(task);
        t1.start();
        t2.start();
    }
}

```

Output -

Thread One : 1
 Thread Two : 3
 Thread One : 2
 Thread Two : 2
 Thread Two : 3
 Thread One : 3
 Thread One : 4
 Thread Two : 4
 Thread One : 5
 Thread Two : 5

Here, we used :
 t1.start();
 t2.start();

not t.run()

t2.run() ; because start() creates a new

thread and then calls run().

5.3 Custom check exception InvalidRadiusException. Circle constructor throws it if $\text{radius} < 0$. Main reads radius, handles via try-catch, else print area.

```
import java.util.Scanner;

class InvalidException extends Exception {
    public InvalidRadiusException (String message) {
        super(message);
    }
}

class Circle {
    private double radius;

    public Circle (double radius) throws InvalidRadiusException {
        if (radius < 0) {
            throw new InvalidRadiusException (
                "Radius cannot be negative."
            );
        }
        this.radius = radius;
    }

    public double getArea() {
        return Math.PI * radius * radius;
    }
}

public class Main {
    public static void main (String[] args) {
        Scanner input = new Scanner (System.in);
        System.out.println ("Enter radius: ");
    }
}
```

```

double radius = input.nextDouble();
try {
    Circle c = new Circle(radius);
    System.out.println("Area = " + c.getArea());
} catch (InvalidRadiusException e) {
    System.out.println("Error: " + e.getMessage());
}
}
}

```

Input :- Enter Radius : 5

Output :- Area = 78.53981633974483

Input: Enter Radius : -5

Output: Error : Radius cannot be negative.

6.1 Steps to connect Java to MySQL/Oracle via JDBC.

Name key classes (DriverManager, Connection, PreparedStatement, ResultSet).

JDBC (Java Database Connectivity) is an API that allows Java programs to communicate with database such as MySQL and Oracle.

Main Steps to connect Java to MySQL -

Step-1 - Add JDBC Driver -

Add the MySQL Connector/J or Oracle JDBC driver to the project.

Step-2 - Load/Register Driver -

Modern JDBC drivers are usually automatically discovered, but traditionally -

`Class.forName ("com.mysql.cj.jdbc.Driver");`

Step 3 - Create Connection -

```
Connection con = DriverManager.getConnection (
    url, username, password );
```

Step 4 - Create Prepared Statement

```
PreparedStatement ps = con.prepareStatement(
    "INSERT INTO Students VALUES (?, ?, ?)" );
```

Step 5 - Execute SQL -

```
ps.executeUpdate();
```

Step 6 - Read Result -

For SELECT :

```
ResultSet rs = ps.executeQuery();
```

Step 7 - Close Resources

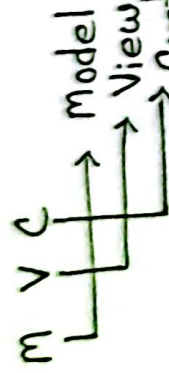
close ResultSet, PreparedStatement and Connection

Important JDBC classes -

- DriverManager → create database connection
- Connection → represents connection with database
- PreparedStatement → executes parameterized SQL
- ResultSet → stores result of SELECT

6.2 Explain the MVC pattern and how a JDBC app's classes map to Model/View/Controller.

MVC stands for:



tion into three parts. MVC separates an applica-

Model - represents data and business information.

Example: class Student {

```

    int id;
    String name;
    double cgpa;
  }
  
```

View - The view is responsible for user interface / input/output.

Example: • JSP • JavaFx • HTML

Controller - The Controller receives user requests and controls the application flow.

for JDBC applications, a DAO (Data Access Object) is often used to perform database operations.

Example:

```

    View
    ↓
    Controller / DAO
    ↓
    Model / Database
  
```

JDBC MVC mapping -

<u>MVC</u>	<u>Example</u>
Model	Student.java
View	Main.java / JSP
Controller/DAO	studentDAO.java
Database	MySQL/Oracle.

Advantages -

MVC makes the application organized, maintainable, reusable and easier to debug.

6.3 MVC-style code for student: Model (Student), Controller / DAO (StudentDAO.insert()) using Prepared Statement(), View (Main) collecting input and calling the controller.

Model-

```
class Student {
    private int id;
    private String name;
    private double cgpa;

    public Student (int id, String name, double cgpa) {
        this.id = id;
        this.name = name;
        this.cgpa = cgpa;
    }

    public int getID() {
        return id;
    }

    public String getName() {
        return name;
    }

    public double getCgpa() {
        return cgpa;
    }
}
```

DAO

```

import java.sql.*;

class StudentDAO {
    private String url =
        "jdbc:mysql://localhost:3306/student_db";

    private String username = "root";
    private String password = "1234";

    public void (Student s) {
        String sql = "INSERT INTO Students (id,
            name, cgpa) VALUES (?, ?, ?)";

        try (
            Connection con = DriverManager.getConnection(
                url, username, password );

            PreparedStatement ps = con.prepareStatement(sql)
        ) {
            ps.setInt(1, s.getId());
            ps.setString(2, s.getName());
            ps.setDouble(3, s.getCgpa());

            ps.executeUpdate();

            System.out.println("Student inserted.");
        }
    }
}

```

```

} catch (SQLException e) {
    System.out.println (e.getMessage());
} } }

```

View/Main -

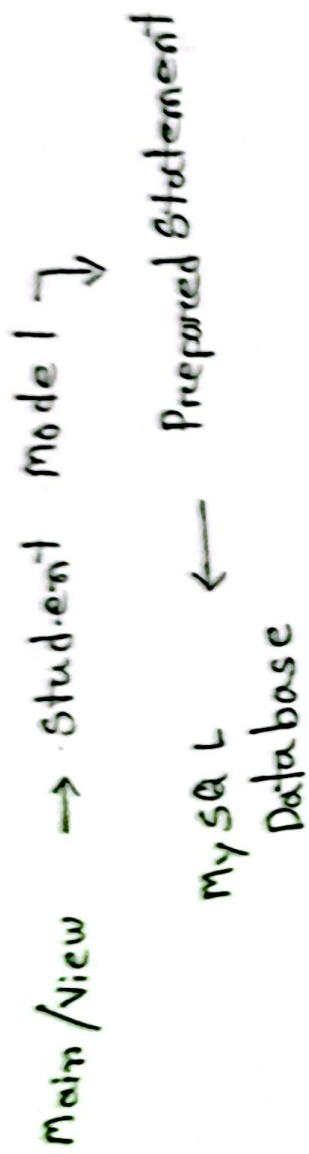
```

import java.util.Scanner;

public class Main {
    public static void main (String[] args) {
        Scanner input = new Scanner (System.in);
        System.out.print ("Enter ID: ");
        int id = input.nextInt();
        input.nextLine();
        System.out.print ("Enter Name: ");
        String name = input.nextLine();
        System.out.print ("Enter CGPA: ");
        double cgpa = input.nextDouble();
        Student student = new Student (id, name, cgpa);
        StudentDAO dao = new StudentDAO();
        dao.insert (student);
    }
}

```

Flow:



7.1 Explain the JavaFX application structure (Application, Stage, Scene, Grid Pane / VBox). Role of start() ?

JavaFX - is a Java framework for creating graphics user interfaces.

A JavaFX application normally extends the Application class.

```
public class Main extends Application {
    @Override
    public void start(Stage stage) {
        // GUI code
    }
}
```

Important components -

• The main class of a JavaFX application extends Application .

• Stage - The stage represents the main window.

• Scene - Contains all GUI components displayed inside the stage.

• Grid Pane - arranges controls in rows and columns

• VBox - arranges components vertically

Role of start() - the start() method is the main entry point for building the JavaFX user interface.

Example -

@Override

```
public void start (Stage stage) {  
    // create GUI
```

```
}
```

7.2 Design the GridPane layout: Loan Amount, Annual Rate, Number of Years inputs + "Calculate" button + result labels.

```
import javafx.application.Application;
import javafx.geometry.Insets;
import javafx.scene.Scene;
import javafx.scene.control.*;
import javafx.scene.layout.GridPane;
import javafx.stage.Stage;

public class LoanCalculator extends Application {
```

@Override

```
public void start(Stage stage) {
    GridPane grid = new GridPane();
    grid.setPadding(new Insets(10));
    grid.setHgap(10);
    grid.setVgap(10);
```

```
Label loanLabel = new Label("Loan Amount:");
TextField loanField = new TextField();

Label rateLabel = new Label("Annual Rate(%)");
TextField rateField = new TextField();

Label yearsLabel = new TextField();
```

```

Button calculateButton = new Button("Calculate");
Label monthlyLabel = new Label("Monthly Payment:");
Label totalLabel = new Label("Difference:");

```

```

grid.add(loanLabel, 0, 0);
grid.add(loanField, 1, 0);
grid.add(rateLabel, 0, 1);
grid.add(rateField, 1, 1);
grid.add(yearsLabel, 0, 2);
grid.add(yearField, 1, 2);
grid.add(CalculateButton, 1, 3);
grid.add(monthlyLabel, 0, 4);
grid.add(totalLabel, 0, 5);
grid.add(differenceLabel, 0, 6);

```

```

Scene scene = new Scene(grid, 400, 300);

```

```

stage.setTitle("House Loan Calculator");

```

```

stage.setScene(scene);

```

```

Stage.show();
}

```

```

public static void main(String[] args) {
    launch(args);
}
}

```

Layout:

Loan Amount: []

Annual Rate(%): []

Number of Years: []

[Calculate]

Monthly Payment:

Total Payment :

Difference :

7.3

Formula -

Let : P = Loan Amount

R = Monthly interest rate

n = Total number of monthly payments

Monthly rate : r = annual Rate / 100 / 12

Number of payments : n = years $\times$ 12

Monthly installment : $M = P \times r \times (1+r)^n / ((1+r)^n - 1)$

Total payment : Total = $M \times n$

Difference : Difference = Total Payment -
Loan Amount

Event handler -

```

calculateButton.setOnClickListener(e -> {
    double loan = Double.parseDouble(
        loanField.getText());

    double annualRate = Double.parseDouble(
        rateField.getText());

    int years = Integer.parseInt(
        yearsField.getText());

    double monthlyRate = annualRate / 100 / 12;

    int months = years * 12;

    double monthlyPayment;

    if (monthlyRate == 0) {
        monthlyPayment = loan / months;
    } else {
        monthlyPayment = loan * monthlyRate *
            math.pow(1 + monthlyRate, months)
            / (math.pow(1 + monthlyRate, months) - 1);
    }
}

```

```
double totalPayment = monthlyPayment * months;  
double difference = totalPayment - loan;  
monthlyLabel.setText(  
    "Monthly Payment: " + monthlyPayment  
);  
totalLabel.setText(  
    "Total Payment: " + totalPayment);  
differenceLabel.setText("Difference: " +  
    difference);  
} );
```

8.1 Compare Socket Programming vs Java RMI . When to prefer each ?

Socket programming and Java RMI , both are techniques for communication between programs running on different machine.

Socket	Java RMI
Lower-level communication	Higher level communication
User IP address and port	User remote objects
Developer handles input/output streams	Java handles much of communication
Can be used between different languages	Mainly java-to-java
More control	Easier object oriented approach
Good for custom chat/network protocols	Good for distributed Java applications

Socket is preferred when -

- Building chat application
- Developing custom client-server protocols.
- Need more control over communication.

RMI is preferred when -

- Both client-server are Java
- We want to call methods of a remote Java object.
- Object-oriented distributed programming is required.

8.2 Server code using ServerSocket: accept connection, read via BufferedReader, reply via PrintWriter.

```
import java.io.*;
import java.net.*;

public class Server {
    public static void main (String[] args) {
        try {
            ServerSocket serverSocket = new ServerSocket(5000);
            System.out.println ("server started...");
            System.out.println ("Waiting for client ...");
            Socket socket = serverSocket.accept ();
            BufferedReader in = new BufferedReader(
                new InputStreamReader(
                    socket.getInputStream()
                )
            );
            PrintWriter out = new PrintWriter(
                socket.getOutputStream(),
                true
            );
            String message = in.readLine();
            System.out.println ("Client: " + message);
            out.println ("Message received by server.");
        }
    }
}
```

```
socket.close();  
serverSocket.close();  
} catch (IOException e) {  
    System.out.println (e.getMessage());  
}}}
```

Main steps -

create ServerSocket



accept() client



Read message



Process message



Send Reply



Close connection.

eg Client code using Socket: connect to server, send typed message, print server reply.

```
import java.io.*;
import java.net.*;
import java.util.Scanner;

public class Client {
    public static void main (String[] args) {
        try {
            Socket socket = new Socket ("localhost", 5000);
            BufferedReader in = new BufferedReader (
                new InputStreamReader (
                    socket.getInputStream ()
                )
            );
            PrintWriter out = new PrintWriter (socket.getOutputStream(),
                true
            );
            Scanner input = new Scanner (System.in);
            System.out.print ("Enter message: ");
            String message = input.nextLine();
            out.println (message);
            String reply = in.readLine ();
```

```
    System.out.println ("Server : " + reply),  
    Socket.close();  
} catch (IOException e) {  
    System.out.println (e.getMessage());  
}  
}  
}.
```

Execution-

Firstly run : Server

Then run : Client

Example : Client : Hello Server

Server: Message received by server

9.4 Client code using Socket : connect to server,
send typed message, print server reply.

Step 1 - Create Database

```
CREAT DATABASE student_db;
```

Step 2 - Select Database

```
USE student_db;
```

Step 3 - Create Table

```
CREAT TABLE Students (
    id INT PRIMARY KEY,
    name VARCHAR (100),
    cgpa DOUBLE
);
```

Step 4 - Add JDBC Driver

Add MySQL Connector/J to the project.

Step 5 - Connection String

```
String url = "jdbc:mysql://localhost:3306/student_db";
```

```
String username = "root";
```

```
String password = "1234";
```

Step 6 - Servlet Mapping

Using annotation :

```
@WebServlet("/student")
```

The request :

/student

will be handled by the servlet.

Basic flow -

HTML/JSP Form



Servlet



JDBC



MySQL



Servlet/JSP Response

9.2 Servlet doPost(): read ID/Name/CGPA from form, insert via PreparedStatement, with exception handling.

```
import java.io.*;
import java.sql.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;
import jakarta.servlet.annotation.*;
```

@WebServlet ("/student").

public class StudentServlet extends HttpServlet {

protected void doPost (

HttpServletRequest request,

HttpServletResponse response)

throws ServletException, IOException {

int id = Integer.parseInt (

request.getParameter ("id")

);

String url = "jdbc:mysql://localhost:3306/student-db";

String username = "root";

String password = "1234";

String sql =

"INSERT INTO Students(id, name, cgpa) +

"VALUES (?, ?, ?)";

```

try {
    Connection con = DriverManager.getConnection(
        url, username, password
    );
    PreparedStatement ps = con.prepareStatement(sql);

    ps.setInt(1, id);
    ps.setString(2, name);
    ps.setDouble(3, gpa);

    ps.executeUpdate();

    response.getWriter().println(
        "Student inserted successfully."
    );

    ps.close();
    con.close();

} catch (SQLException e) {
    response.getWriter().println(
        "Database Error:" + e.getMessage()
    );
}
}
}

```

important: Prepared statement is preferred because it -

- Users permanent
- makes SQL easier to handle
- Helps protect against SQL injection

9.3 JSP page: connect to student-db, SELECT from Students, display as HTML table.

```
<%@ page import = "java.sql.*" %>
```

```
<html>
```

```
<head>
```

```
<title> Student List </title>
```

```
</head>
```

```
<body>
```

```
<h2> Student Records </h2>
```

```
<table border = "1" >
```

```
<tr>
```

```
<th> ID </th>
```

```
<th> Name </th>
```

```
<th> CGPA </th>
```

```
</tr>
```

```
<%
```

```
String url = "jdbc:mysql://localhost:3306/student-db";
```

```
String username = "root";
```

```
String password = "1234";
```

```
String sql = "SELECT id, name, cgpa FROM Students";
```

```
try {
```

```
Connection con = DriverManager.getConnection(
    url, username, password
```

```

);
PreparedStatement ps = con.prepareStatement(sql);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    // >
}
rs.close();
ps.close();
con.close();
} catch (SQLException e) {
    out.println (
        "Error: " + e.getMessage()
    );
}
// >
</table>
</body>
</html>

```

Output concept :

ID	Name	CGPA
1	Rahim	3.50
2	Patul	3.75
3	Raha	3.90